Proyecto

NERVIO

Simulador de entrevistas con IA y voz en tiempo real

FASE 3 — Implementación y Despliegue del MVP (Sprint 1)

Docente

Silvia Janeth Rico De Serrano

Asignatura

Aplicación De Técnica De Ingeniería De Software — Grupo 01

Estudiantes

Cornejo Raymundo, Grecia Michelle — CR100723

Cruz García, Melanie Adriana — CG100223

Coreas Hernández, Fátima Sarahi — CH100723

Esquivel Ramírez, Jesús Gerardo — ER100523

Mejía Reinosa, Ronald Eduardo — MR100823

Orellana Morán, Oscar Alejandro — OM100123

Fecha: 31 de octubre de 2026

Contenido

Actualiza este índice en Word: clic derecho → Actualizar campos.

1. Objetivo de la entrega

Esta entrega corresponde a la Fase 3 del proyecto Nervio: traducir la especificación técnica, la arquitectura y los prototipos definidos en la Fase 2 (documento maestro Actividad evaluada N.° 4) en un incremento de software funcional y ejecutable (MVP), aplicando arquitectura en capas, seguridad en la capa de aplicación, persistencia de datos e interacción por voz en tiempo real. El alcance del Sprint 1 abarca acceso seguro (registro, inicio de sesión y recuperación de contraseña), configuración y arranque de la entrevista, ejecución conversacional con IA, carga del CV para personalizar la entrevista, generación de reporte, historial, agendamiento, grabaciones, enlaces para compartir reportes, roles con panel de administración (gestión de usuarios) y un módulo de soporte con actualizaciones en tiempo real.

Las pruebas automatizadas (unitarias, integración y E2E) no forman parte de esta fase y se abordarán en la Fase 4 (Calidad de Software); aquí se documentan pruebas manuales y exploratorias.

2. Repositorio de código fuente y estrategia de ramas

El código se gestiona en un repositorio Git (GitHub) con historial activo y distribuido: más de 50 commits en develop de varios integrantes. Nervio es una sola aplicación full-stack en la carpeta frontend (Next.js con App Router): la API (Route Handlers en src/app/api) y la interfaz conviven en el mismo proyecto. No existe un servidor backend separado: la antigua carpeta backend fue eliminada del repositorio porque toda la aplicación full-stack vive en frontend/.

2.1. Estrategia de ramas

Rama	Propósito
main	Rama estable; solo recibe develop cuando una versión está lista para evaluar.
develop	Rama de integración; todas las ramas de característica entran aquí por pull request.
feature/roles, feature/user-management	Roles user/admin y panel de gestión de usuarios.
feature/support, feature/realtime	Módulo de soporte (tickets) y actualizaciones en tiempo real por SSE.
feature/api-collection	Colección Postman y verificación pnpm check:api.
feature/demo-readiness, feat/report-transcript	Datos de demo, transcripción y feedback por respuesta.
feature/cv-upload	Carga de CV con miniatura y límite de 20 minutos por entrevista.
agenda, frontend, docs/…	Ramas históricas del Sprint 1 (agenda, interfaz y documentación).

Flujo de trabajo: cada cambio se desarrolla en una rama feature/…, se abre un pull request hacia develop, se revisa y se integra con merge (--no-ff, sin reescribir historial ni force push). develop se integra a main para cada entrega. Las convenciones (nombres de rama, mensajes de commit, revisión)

están en CONTRIBUTING.md. Este esquema mantiene un historial legible por funcionalidad y permite revertir una característica completa si fuera necesario.

2.2. README y puesta en marcha

Prerrequisitos y versiones (obligatorios):

- **Node.js:** ≥ 20 (v20 o superior; probado en 20/22/24). Runtime de Next.js 16 / React 19.
- **pnpm:** ≥ 9 (v9 o superior). Gestor de paquetes del proyecto (existe pnpm-lock.yaml).
- **PostgreSQL:** 15 o 16. Base de datos relacional (Supabase gestionado o instancia local).

El README del proyecto documenta los prerrequisitos y versiones, la instalación, la estructura de variables de entorno (.env.example), la creación de la base de datos mediante migraciones Prisma, la carga de datos de prueba (seed) y el despliegue. Comandos principales:

```
cd frontend
pnpm install          # instala dependencias y ejecuta prisma generate
cp .env.example .env     # completar credenciales
pnpm db:migrate         # aplica el esquema (DDL) -> prisma migrate deploy
pnpm db:seed           # usuarios y datos de prueba (idempotente)
pnpm dev              # http://localhost:3000
```

Además del README de frontend, docs/DEMO.md contiene el guion de la demostración, el plan B y una lista de verificación previa. El seed crea también una cuenta de administrador (ver 5.2).

Estructura de variables de entorno (archivo .env.example):

```
# PostgreSQL (Supabase) — Settings → Database → Connection string
# Runtime (Transaction pooler, puerto 6543). Usuario: postgres.[PROJECT_REF]
POSTGRES_URL="postgresql://postgres.[PROJECT_REF]:[PASSWORD]@aws-0-
[REGION].pooler.supabase.com:6543/postgres?pgbouncer=true&sslmode=require"
# Opcional: si no defines POSTGRES_URL, la app usa DATABASE_URL (cuidado con JDBC en el sistema)
DATABASE_URL="postgresql://postgres.[PROJECT_REF]:[PASSWORD]@aws-0-
[REGION].pooler.supabase.com:6543/postgres?pgbouncer=true&sslmode=require"
# Migraciones Prisma (Direct connection, puerto 5432). Usuario: postgres
DIRECT_URL="postgresql://postgres:[PASSWORD]@db.[PROJECT_REF].supabase.co:5432/postgres?
sslmode=require"

# Better Auth
BETTER_AUTH_SECRET="generar-con-openssl-rand-base64-32"
BETTER_AUTH_URL="http://localhost:3000"
NEXT_PUBLIC_BETTER_AUTH_URL="http://localhost:3000"

# ElevenLabs Conversational AI (solo servidor — no usar NEXT_PUBLIC_)
ELEVENLABS_API_KEY="tu-api-key"
ELEVENLABS_AGENT_ID="tu-agent-id"
ELEVENLABS_VOICE_ID="tu-voice-id"

# N8N opcional: si faltan o fallan, Next.js usa Prisma/local fallback
```

```
N8N_WEBHOOK_GENERATE_INTERVIEW_URL=""
N8N_WEBHOOK_SCHEDULE_NOTIFICATION_URL=""

# URL pública de la app (enlaces en correos de citas)
NEXT_PUBLIC_APP_URL="http://localhost:3000"

# Resend — confirmación al agendar y recordatorio según el margen
RESEND_API_KEY=""
EMAIL_FROM="Nervio <onboarding@resend.dev>"

# Cron de Vercel: Authorization Bearer de /api/cron/reminders
CRON_SECRET=""

# Mock local sin agente ElevenLabs (opcional)
# NEXT_PUBLIC_USE_MOCK_INTERVIEW="true"
```

3. Arquitectura implementada

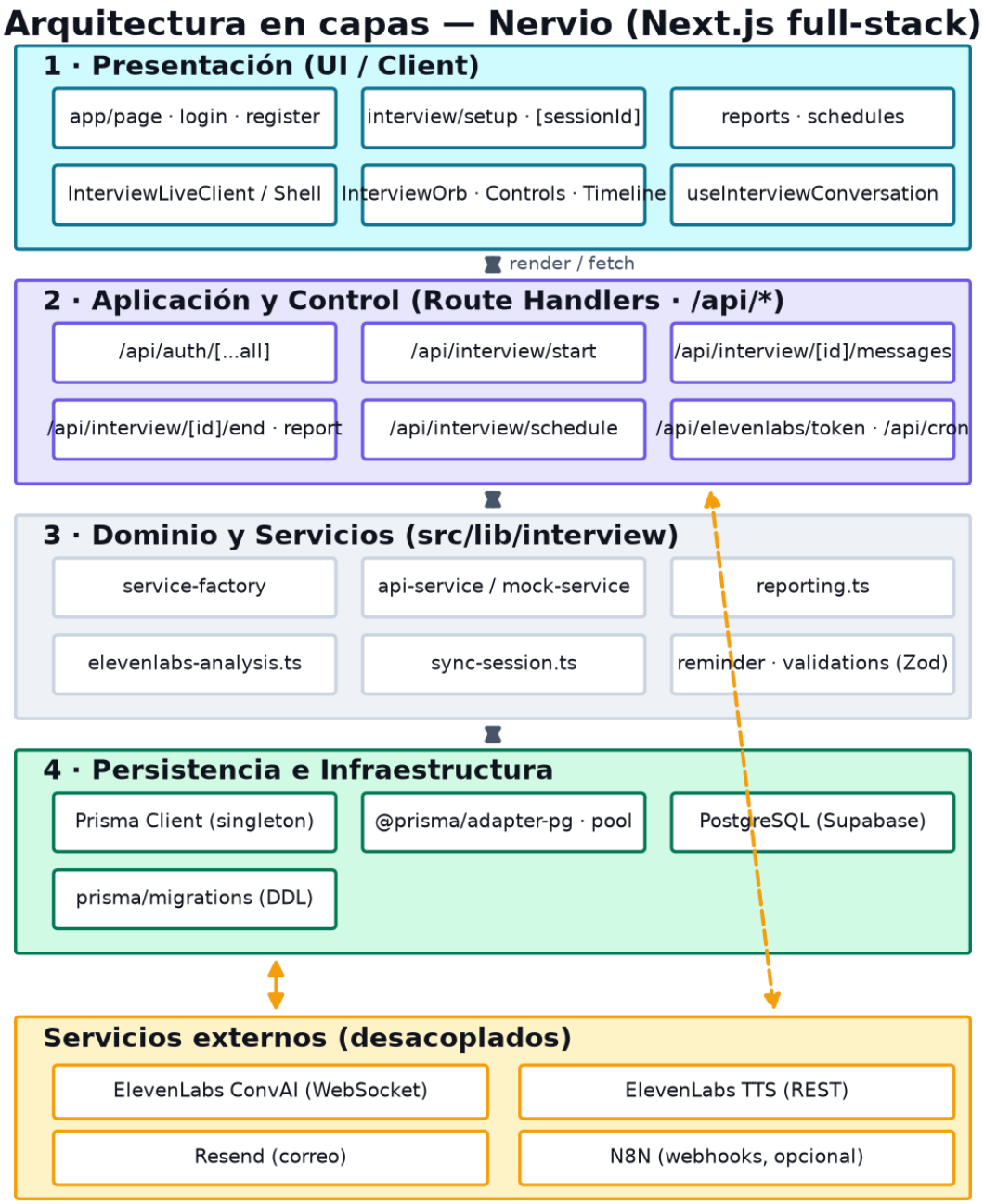
Nervio implementa una arquitectura Cliente-Servidor en capas con un diseño de monolito modular full-stack sobre Next.js (App Router, React 19 y Node.js), desacoplado de servicios externos especializados: ElevenLabs para voz en tiempo real y, opcionalmente, N8N para tareas asíncronas. El sistema se organiza en cuatro capas:

- **Capa de Presentación (UI/Client):** Server y Client Components de React, Tailwind CSS y componentes shadcn/base-ui; captura de audio, orbe reactivo, línea de tiempo y formularios.
- **Capa de Aplicación y Control (Route Handlers, /api/*):** valida con Zod, verifica autenticación y propiedad de la sesión y coordina los servicios.
- **Capa de Dominio y Servicios (src/lib/interview):** ciclo de vida de la sesión, métricas, normalización de preguntas/respuestas y lógica de fallback.
- **Capa de Persistencia e Infraestructura:** Prisma Client con pool de conexiones (@prisma/adapter-pg) sobre PostgreSQL y control de migraciones.

Servicios externos: ElevenLabs (voz y agente conversacional), Supabase PostgreSQL y Storage (CV y grabaciones), Resend (correos) y Vercel Cron (recordatorios y limpieza de grabaciones). N8N es opcional y tiene alternativa local.

Figura 1

Arquitectura en capas de la plataforma Nervio.



Nota. Representa las cuatro capas lógicas y su relación con los servicios externos desacoplados. Fuente: elaboración propia a partir del repositorio del proyecto.

4. Backend: API y comunicación en tiempo real

El backend se implementa como Route Handlers de Next.js (src/app/api/*) que actúan como controladores bajo el patrón Controller–Service–Repository. Cada endpoint valida la sesión, el rol

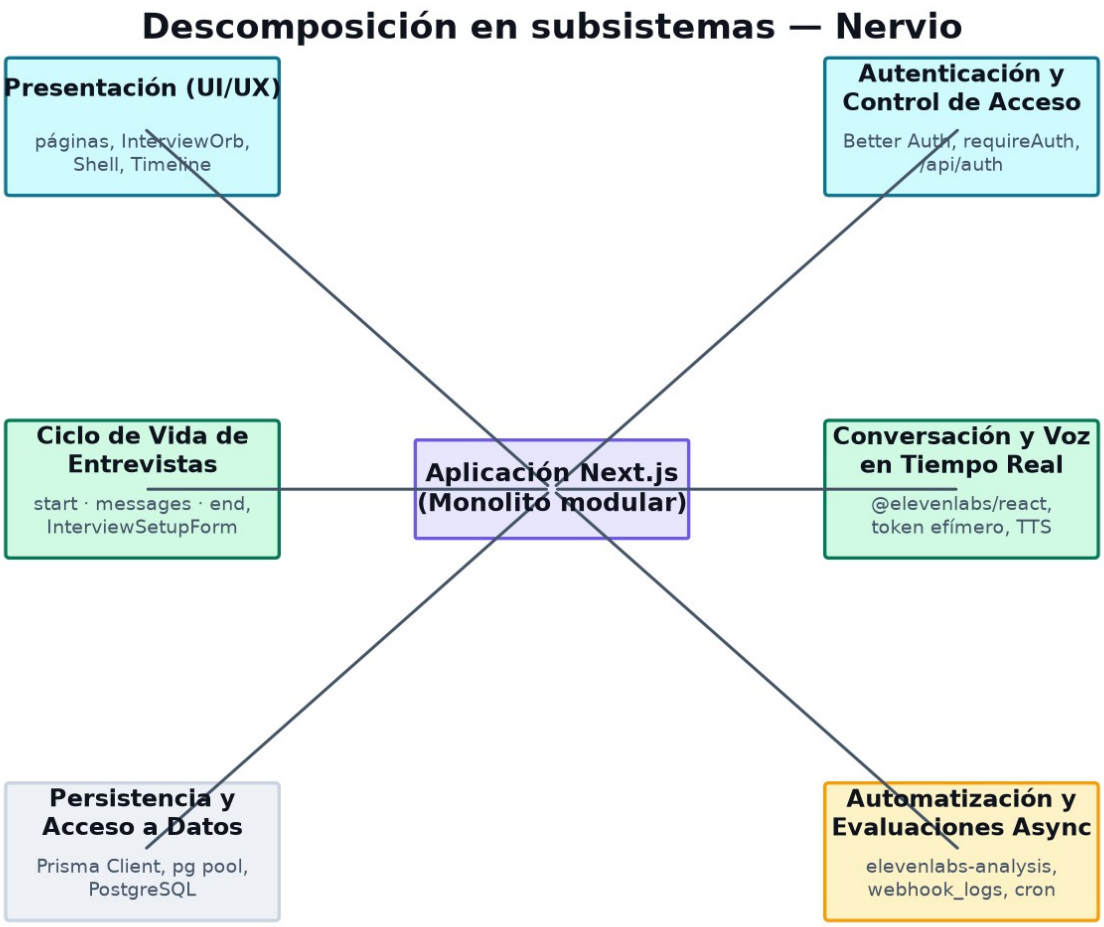
cuando aplica y la propiedad del recurso antes de operar. Catálogo completo (cubre autenticación, gestión de usuarios, recursos principales y soporte):

4.1. Catálogo de endpoints

Método	Endpoint	Auth	Respuesta principal
*	/api/auth/[...all]	Pública/Sesión	Registro, login, logout, recuperación de contraseña (Better Auth)
GET/PATCH	/api/users/me	Requerida	Perfil del usuario autenticado
GET/PATCH	/api/admin/users	Rol admin	Listar, buscar, cambiar rol, banear/desbanear
POST/GET	/api/cv, /api/cv/[cvId]	Requerida (dueño)	Subir CV (PDF, máx. 5 MB), listar, ver, eliminar
GET	/api/cv/[cvId]/file	Requerida (dueño)	Archivo del CV para la miniatura
POST	/api/interview/start	Requerida	sessionId, preguntas
GET	/api/interview/[id]	Requerida (dueño)	Sesión con preguntas y respuestas
POST	/api/interview/[id]/ messages	Requerida (dueño)	Registro de turnos
POST	/api/interview/[id]/end	Requerida (dueño)	{ ended, reportReady }
GET	/api/interview/[id]/report, /report/pdf	Requerida (dueño)	Reporte y PDF
GET	/api/interview/[id]/ recording, /recording/audio	Requerida (dueño)	Grabación (14 días)
POST	/api/interview/[id]/share; GET /api/r/[token]/pdf	Dueño / Token	Enlace público de solo lectura
GET	/api/interview/reports	Requerida	Historial del usuario
POST/PATCH	/api/interview/schedule, /schedule/[id]	Requerida (dueño)	Crear y editar citas
POST/GET	/api/support, /api/support/[id], /replies	Requerida (dueño)	Tickets de soporte y respuestas
GET/PATCH	/api/admin/support, /api/admin/support/[id]	Rol admin	Bandeja, estado, prioridad, respuesta
GET	/api/realtime/support/ [id], /api/realtime/interview/[id]	Requerida	Flujo SSE de eventos
GET	/api/elevenlabs/token	Requerida	Token efímero para la sesión de voz
GET	/api/cron/reminders, /api/cron/recordings-cleanup	Bearer CRON_SECRET	Recordatorios y borrado de grabaciones vencidas

Figura 2

Descomposición en subsistemas de la plataforma Nervio.



Nota. Seis subsistemas de alta cohesión coordinados por la aplicación Next.js. Fuente: elaboración propia.

4.2. Mecanismos de seguridad activos

- **Hashing de contraseñas:** Better Auth (scrypt con sal aleatoria); nunca se almacena texto plano (tabla account.password).
- **Sesiones por cookie:** cookies HttpOnly, Secure y SameSite=Lax (mitigan XSS y CSRF).
- **Control de propiedad (anti-IDOR):** cada ruta /api/interview/[sessionId] valida que interviewSession.userId coincida con el usuario autenticado.
- **Validación de entradas con Zod:** todo payload se valida antes de llegar al dominio o a la base de datos (HTTP 400 ante datos inválidos).
- **Custodia de secretos:** ELEVENLABS_API_KEY, BETTER_AUTH_SECRET y DATABASE_URL residen solo en el servidor (sin prefijo NEXT_PUBLIC_).
- Límite de intentos: respuestas 429 ante exceso de peticiones en rutas sensibles.

- Archivos: el CV solo se sirve a su dueño (401 sin sesión, 404 para otros usuarios); tamaño máximo 5 MB (413).

Modelo de roles y permisos. Nervio tiene dos roles: user (candidato) y admin, implementados con el plugin admin de Better Auth (columnas role, banned, banReason y banExpires en la tabla user; migración admin_roles). Las rutas /api/admin/* y las páginas /admin exigen el rol admin (src/lib/require-role.ts) y responden 403 «Prohibido: permisos insuficientes» si no se cumple. El administrador puede listar y buscar usuarios, cambiar su rol, banearlos y gestionar los tickets de soporte. Además, el control de propiedad (anti-IDOR) garantiza que cada candidato solo vea y opere sus propias entrevistas, CV, reportes, citas y tickets. El seed crea una cuenta por rol (apartado 5.2).

4.3. Manejo de errores (respuestas HTTP semánticas)

Código	Significado	Ejemplo en Nervio
400	Datos inválidos	Payload que no pasa el esquema Zod.
401	No autenticado	Petición sin sesión válida.
403	Sin permisos	Usuario sin rol admin en /api/admin/*.
404	No encontrado	Sesión, reporte o CV inexistente o de otro usuario.
409	Conflicto	Reporte aún no listo; ticket cerrado al responder.
410	Recurso expirado	Grabación vencida tras 14 días.
413	Archivo demasiado grande	CV de más de 5 MB.
429	Demasiadas peticiones	Límite de intentos superado.
500	Error interno	Fallo inesperado en el controlador.
502/503	Fallo de servicio externo	Error del webhook de generación o servicio no disponible.

4.4. Comunicación en tiempo real

Nervio usa dos mecanismos de tiempo real, cada uno elegido por su flujo. (1) Voz bidireccional: el SDK @elevenlabs/react abre una conexión WebRTC/WebSocket directa entre el navegador y el agente de ElevenLabs para streaming de audio de baja latencia en ambos sentidos; el servidor solo emite un token efímero (/api/elevenlabs/token) y nunca expone la clave. El hook useInterviewConversation gestiona los estados (conectando, hablando, escuchando, finalizando), sincroniza la transcripción y detecta la pérdida de conexión.

(2) Eventos del servidor por SSE (Server-Sent Events): /api/realtime/support/[ticketId] y /api/realtime/interview/[sessionId] envían al cliente las respuestas y cambios de estado de un ticket y el estado de la entrevista (por ejemplo, cuando el reporte queda listo) sin recargar la página (src/lib/realtime). Se eligió SSE porque estos flujos van solo del servidor al cliente, funcionan sobre HTTP en Vercel sin infraestructura extra y se reconectan solos; las acciones del cliente siguen usando peticiones REST.

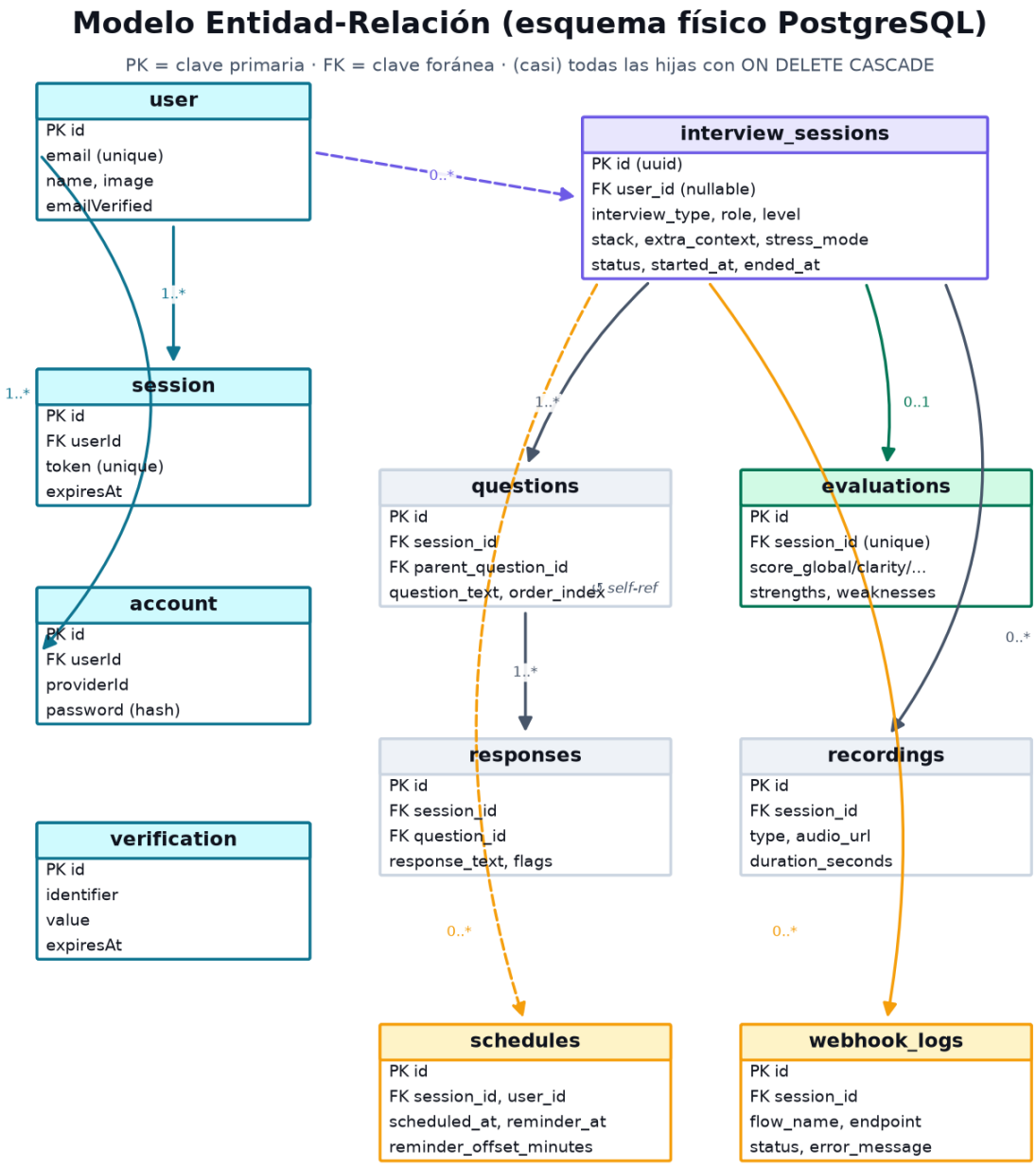
Límite de duración: la entrevista dura como máximo 20 minutos. La pantalla muestra el tiempo restante; a los 2 minutos la app avisa al candidato y envía un mensaje de contexto al agente para que lo anuncie en voz alta, a los 30 segundos le pide despedirse y la app cierra la llamada de forma ordenada. Un cierre por el agente o por duración se trata como fin normal, no como error de conexión.

5. Base de datos y persistencia

La persistencia se implementa sobre PostgreSQL (Supabase) mediante Prisma ORM. El esquema (prisma/schema.prisma) define tres núcleos: autenticación y roles (user, session, account, verification), dominio de entrevistas (interview_sessions, questions, responses, evaluations, recordings, schedules, CV del candidato, enlaces de reporte compartido, webhook_logs) y soporte (support_tickets, support_replies).

Figura 3

Modelo Entidad-Relación (esquema físico) de Nervio en PostgreSQL.



Nota. Las entidades subordinadas a una entrevista aplican integridad referencial con eliminación en cascada (onDelete: Cascade). Fuente: elaboración propia a partir de schema.prisma.

5.1. Migraciones (DDL) e integridad

- DDL versionado: las migraciones Prisma (prisma/migrations/) son los scripts de creación del esquema, en orden: init, reminder_lead, reminder_offset_minutes, report_transcript,

recordings_storage, report_shares_reminder_id, admin_roles y support_module. Las migraciones posteriores al init son aditivas.

- **Integridad referencial:** relaciones hijas con onDelete: Cascade; la autorreferencia de questions (parent_question_id) modela follow-ups.
- **Pool de conexiones:** Prisma Client con @prisma/adapter-pg y patrón singleton (src/lib/prisma.ts) para reutilizar conexiones y evitar saturación.

```
pnpm db:migrate        # prisma migrate deploy (aplica el DDL)
pnpm exec prisma migrate dev   # en desarrollo, genera y aplica migraciones
```

5.2. Datos de prueba (seed)

El script prisma/seed.ts crea usuarios de prueba con la API de servidor de Better Auth (hash compatible con el login real), asigna su rol e inserta entrevistas finalizadas con preguntas, respuestas, evaluación, transcripción y una cita futura. Es idempotente (pnpm db:seed).

Usuario	Correo	Contraseña
Candidata Demo (user)	demo@nervio.dev	Nervio#2026
Candidato QA (user)	qa@nervio.dev	Nervio#2026
Administrador (admin)	admin@nervio.dev	Nervio#2026

6. Frontend / Interfaz de usuario

Las pantallas del MVP implementan los prototipos de la Fase 2 (modo oscuro, acentos cian/violeta, tipografía Sora) e integran la API y los eventos en tiempo real. Manejo de estados de la interfaz:

- Cargando: botones deshabilitados con indicador y esqueletos mientras se consulta la API; orbe en estado «conectando».
- Validación: formularios con react-hook-form + Zod y mensajes junto a cada campo (registro, setup, soporte, CV de más de 5 MB o formato no PDF).
- Éxito: notificaciones (toasts), miniatura del CV subido y vista previa del texto extraído, pantalla de fin de entrevista.
- Errores: mensajes del servidor visibles, página 404 real, aviso de mala conexión y de tiempo restante en la entrevista.

Pantalla	Ruta	Función
Landing	/	Propuesta de valor y acceso.
Registro / Login	/register, /login	Alta y acceso con validación.
Recuperar contraseña	/forgot-password, /reset-password	Envío y uso del enlace de un solo uso.
Perfil	/profile	Datos del usuario.
Configurar entrevista	/interview/setup	Rol, nivel, stack, tipo, modo estrés y CV con miniatura.
Entrevista en vivo	/interview/[sessionId]	Orbe, transcripción, controles y tiempo restante.
Fin de entrevista	/interview/[sessionId]/ended	Cierre normal o por tiempo máximo.

Reporte	/interview/[sessionId]/report	Puntajes, feedback, grabación, PDF y compartir.
Reporte compartido	/r/[token]	Vista pública de solo lectura.
Historial	/interview/reports	Entrevistas finalizadas.
Agenda	/interview/schedules	Citas y recordatorios.
Soporte	/support, /support/new, /support/[id]	Tickets del usuario en tiempo real.
Administración	/admin, /admin/support	Usuarios, roles, baneos y bandeja de soporte.

Las siguientes figuras muestran las pantallas implementadas (evidencia de ejecución).

Figura 4

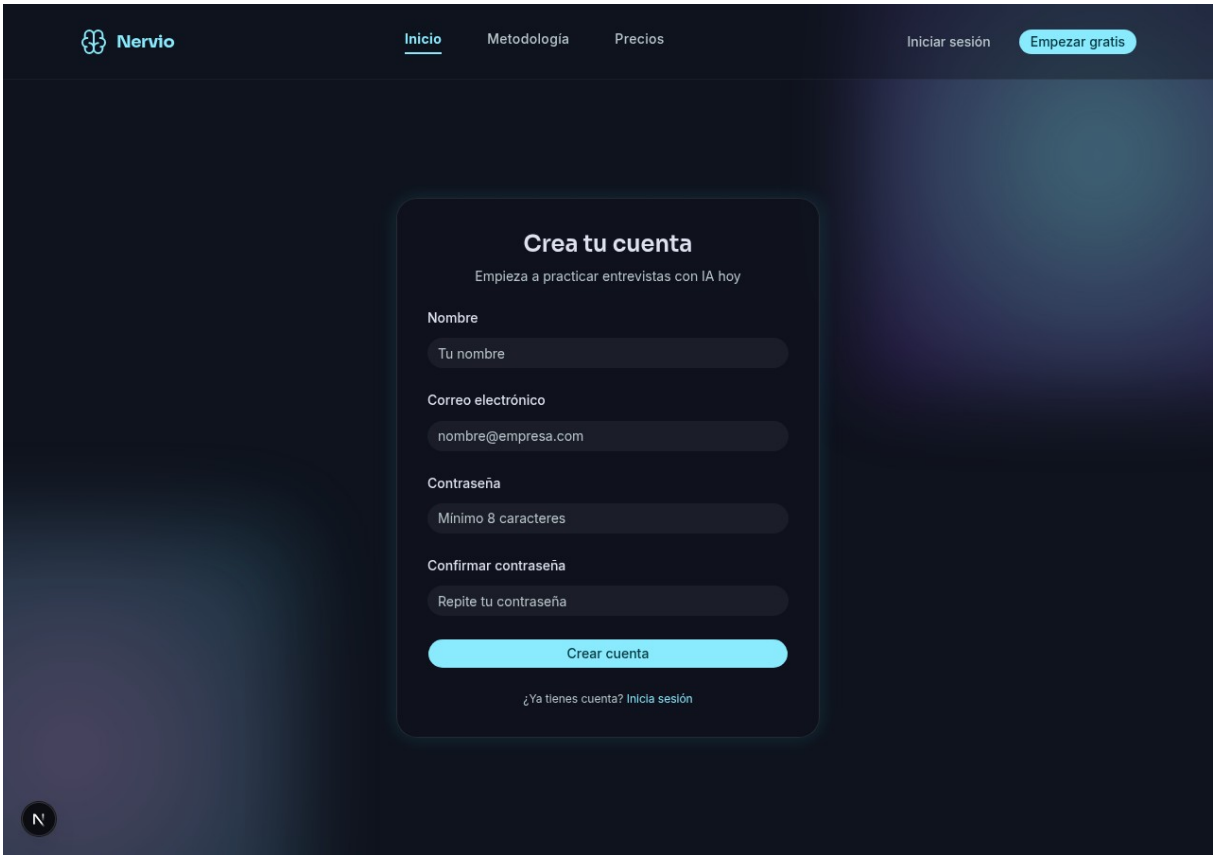
Página principal (landing) de Nervio.



Nota. Pantalla de bienvenida con la propuesta de valor y el acceso al flujo de entrevista.

Figura 5

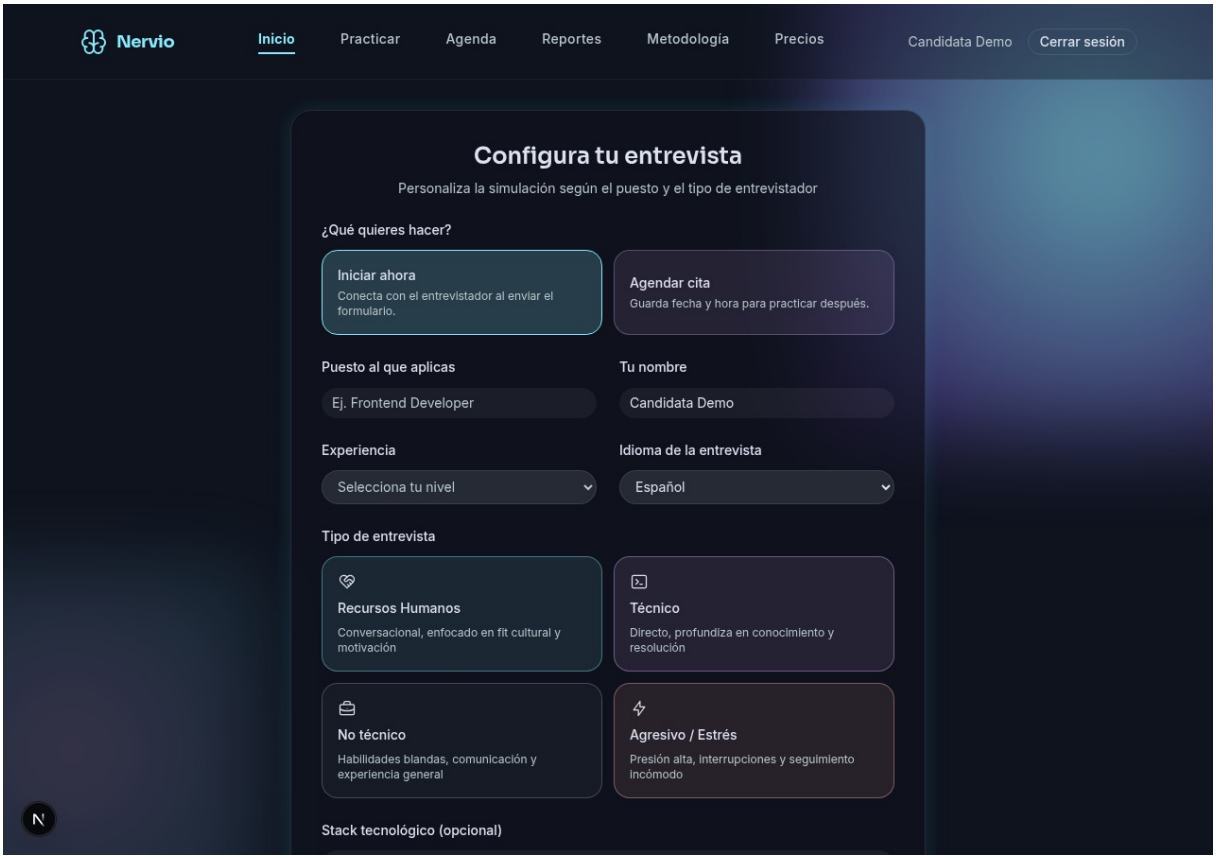
Registro e inicio de sesión del candidato.



Nota. Formularios de alta y acceso con validación de campos.

Figura 6

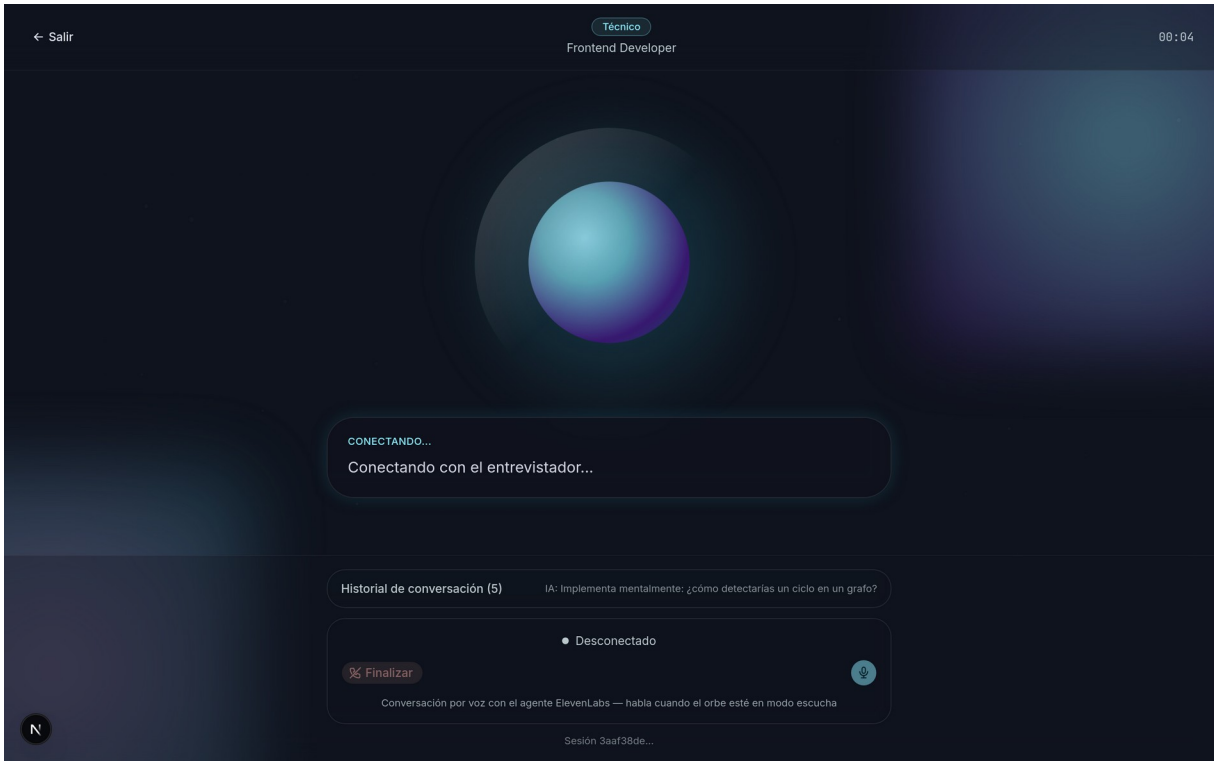
Configuración del contexto de entrevista.



Nota. Formulario de parámetros: tipo, rol, seniority, stack y modo estrés.

Figura 7

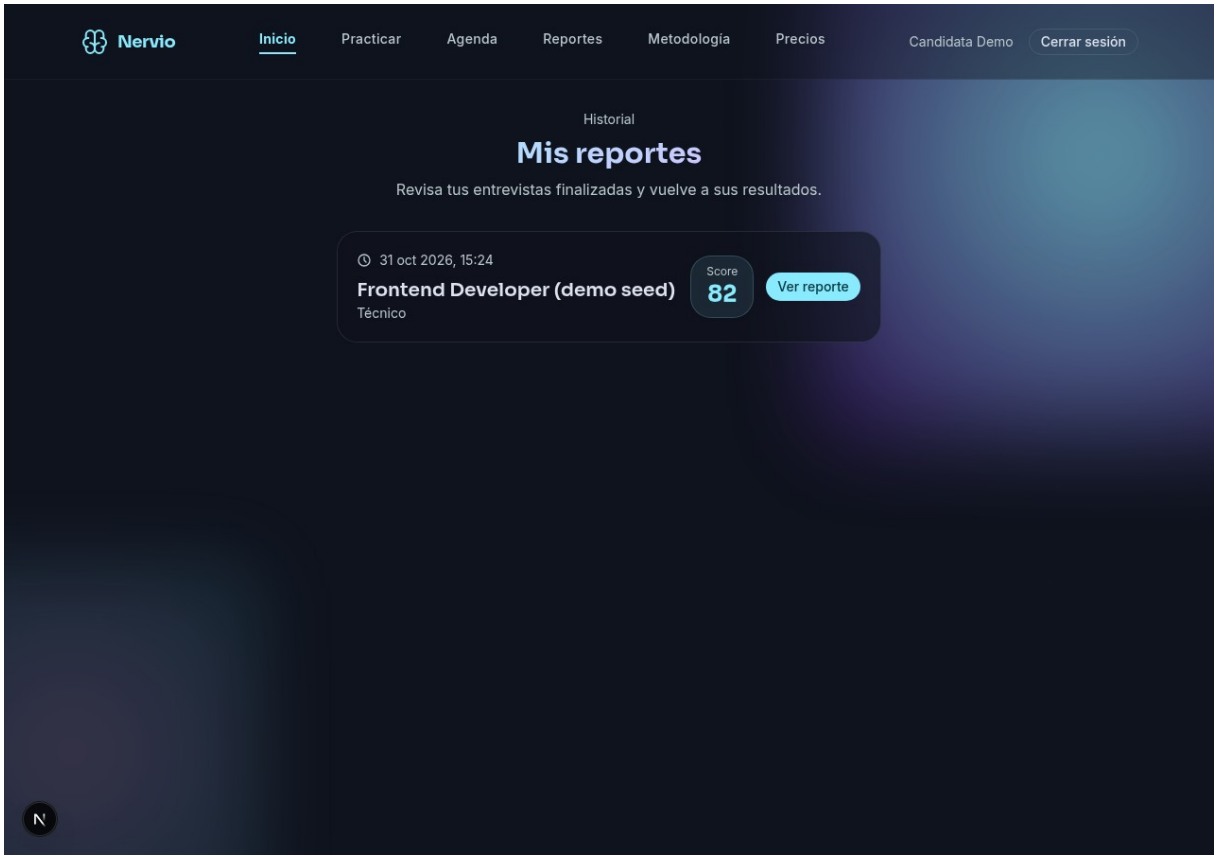
Entrevista en vivo (orbe reactivo y controles).



Nota. Entorno inmersivo de la entrevista en vivo: orbe reactivo de la IA, panel de pregunta y estado de la sesión, historial de conversación y controles de micrófono.

Figura 8

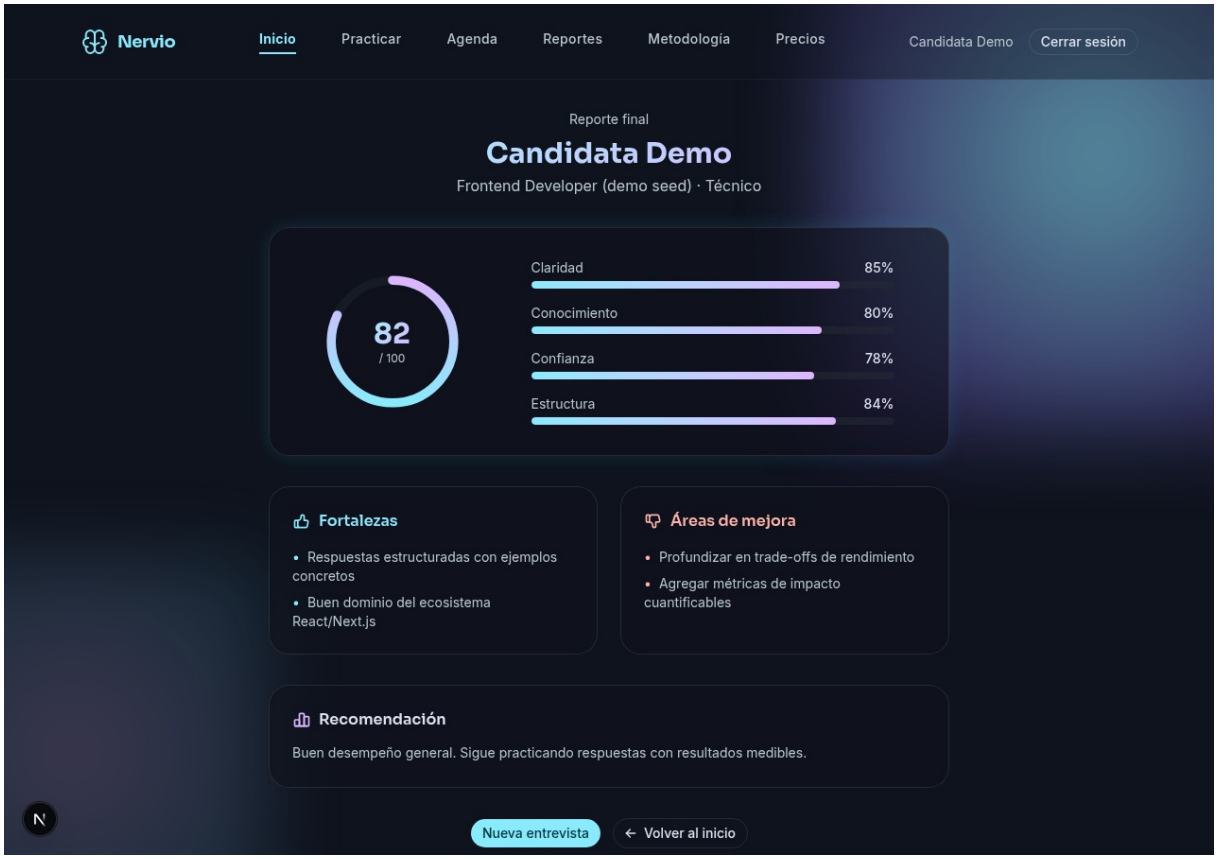
Historial de entrevistas (Mis reportes).



Nota. Listado de entrevistas finalizadas del candidato con su puntuación global.

Figura 9

Reporte de desempeño detallado.



Nota. Puntuación global y por categoría (claridad, conocimiento, confianza, estructura), fortalezas, áreas de mejora y recomendación.

[Pendiente del equipo: agregar capturas de configuración con CV, tiempo restante, soporte en tiempo real y panel de administración.]

7. Colección de pruebas manuales de la API

Se incluye nervio-api.postman_collection.json con su entorno nervio-api.postman_environment.json (Postman v2.1, importable en Postman, Thunder Client o Insomnia): 43 peticiones en 10 carpetas que prueban cada endpoint por separado. La autenticación es por cookie: tras Auth → Sign In el cliente guarda la cookie. El script pnpm check:api verifica que todas las rutas de la API tengan su petición en la colección (resultado actual: 43 peticiones, 0 faltantes). Flujo sugerido: Sign In → CV → Start → Messages → End → Report → Soporte; con admin@nervio.dev, carpeta Admin.

Carpeta	Petición	Verifica
Auth (4)	Sign Up / Sign In / Get Session / Sign Out	Registro, login, sesión y cierre.
Usuarios (2)	Get / Update me	Perfil propio.
Interview (12)	Start, Messages, Get, End, Report, PDF,	Ciclo de vida de la entrevista.

	List, Recording, Share…	
CV (5)	Subir / Listar / Ver / Archivo / Eliminar	Carga y acceso solo del dueño.
Schedule (2)	Create / Update	Agendamiento.
Integraciones (3)	Token ElevenLabs, crons	Token efímero y tareas programadas.
Soporte (4)	Crear, listar, ver, responder	Tickets del usuario.
Tiempo real (SSE) (2)	Stream soporte / entrevista	Eventos del servidor.
Admin (7)	Usuarios, rol, ban/unban…	Requiere rol admin (403 si no).
Admin soporte (2)	Bandeja / actualizar ticket	Gestión de soporte.

8. Pruebas manuales / exploratorias realizadas

Verificación operativa mediante la interfaz y la colección de API (sin código de prueba). Además de los casos de uso del Sprint 1, se registran verificaciones concretas realizadas por el equipo:

Caso de uso	Pasos (resumen)	Resultado esperado
CU-01 Registrar cuenta	Completar registro con datos válidos	Cuenta creada y sesión iniciada.
CU-02 Iniciar sesión	Login con credenciales del seed	Acceso al panel del candidato.
CU-04 Configurar entrevista	Elegir tipo, rol, nivel y stack	Configuración válida lista.
CU-05 Iniciar entrevista	Crear sesión desde el setup	Sesión in_progress y canal listo.
CU-06 Entrevista interactiva	Intercambio de turnos por voz/texto	Preguntas y respuestas persistidas.
CU-07 Finalizar y reporte	Finalizar la sesión	Sesión ended y reporte asociado.
CU-08 Revisar reporte	Abrir el reporte de una sesión	Scores y feedback correctos.
CU-09 Consultar historial	Abrir el historial	Listado propio y ordenado.
CU-10 Agendar entrevista	Crear una cita futura	Cita creada y correo de confirmación.
CU-03 Recuperar contraseña	Solicitar enlace y definir nueva clave	Enlace de un solo uso; login con la nueva clave.
CV: subida y miniatura	Subir un PDF de prueba en el setup	Miniatura de la primera página y texto extraído.
CV: acceso solo del dueño	GET /api/cv/[id]/file como dueño, otro usuario y sin sesión	200 (mismo archivo) / 404 / 401.
Roles	Abrir /admin y /api/admin/users como user	Acceso denegado (403); como admin, listado correcto.
Soporte en tiempo real	Responder un ticket como admin con el usuario en otra ventana	La respuesta aparece sin recargar (SSE).
Límite de 20 minutos	Revisión de una sesión cortada por duración; ajuste del agente y de la app	Aviso a 2 min, despedida y pantalla «Se alcanzó el tiempo máximo».
Colección completa	pnpm check:api	43 peticiones, 0 faltantes.
Calidad de compilación	pnpm lint, tsc y pnpm build	Sin errores.

Nota. Se validó también el control de propiedad: un usuario no accede a sesiones ni reportes de otro (HTTP 404/401).

8.1. Guía de demostración en vivo

El guion completo está en docs/DEMO.md (10 minutos). Orden: repositorio y ramas → README → autenticación y roles → gestión de usuarios → entrevista con CV y voz → reporte e historial → soporte → tiempo real con dos ventanas → errores y validaciones → Postman y pnpm check:api.

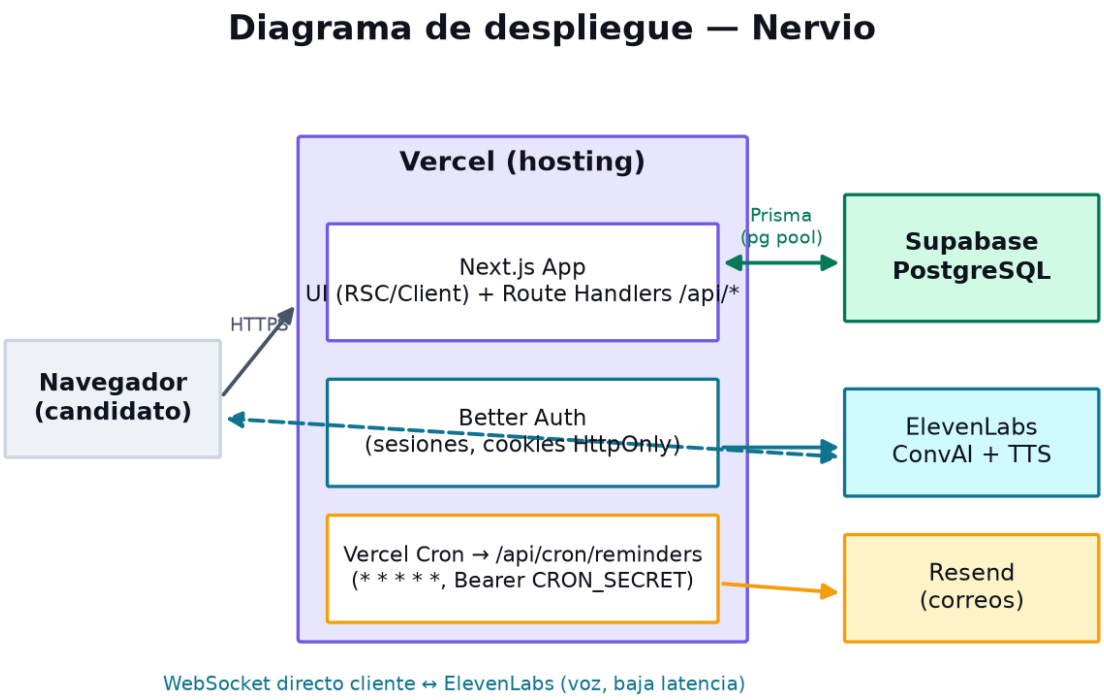
Plan B: si fallan ElevenLabs o el micrófono, activar NEXT_PUBLIC_USE_MOCK_INTERVIEW o mostrar los reportes sembrados; si falla la red, ejecutar en local con pnpm build && pnpm start.

9. Despliegue

La aplicación se despliega en Vercel con base de datos PostgreSQL gestionada (Supabase). El build ejecuta prisma generate && next build. Un Vercel Cron invoca /api/cron/reminders cada minuto (protegido por CRON_SECRET) para enviar recordatorios.

Figura 10

Diagrama de despliegue de Nervio.



Nota. Topología de ejecución: navegador, Vercel (app y cron), Supabase PostgreSQL y servicios externos (ElevenLabs, Resend). Fuente: elaboración propia.

- **Variables de entorno:** DIRECT_URL para migraciones; POSTGRES_URL en runtime; secretos de Better Auth, ElevenLabs, Resend y CRON_SECRET.
- **Base de datos:** ejecutar prisma migrate deploy contra DIRECT_URL en el pipeline.
- **Cron:** definido en vercel.json (schedule * * * * *).

- Crons adicionales: /api/cron/recordings-cleanup elimina las grabaciones con más de 14 días.

10. Trazabilidad Fase 2 → Fase 3

Correspondencia entre los requerimientos/diseño de las fases previas y su implementación.

Requisito (Fase 1/2)	Componente implementado (Fase 3)	Estado
NER-6 / CU-01 Registro	register/page.tsx, Better Auth, tabla user/account	Implementado
NER-7 / CU-02 Login	login/page.tsx, cookie de sesión HttpOnly	Implementado
NER-8 / CU-03 Recuperación	verification, flujo de token de un solo uso, /forgot-password, /reset-password	Implementado
NER-9 / CU-04 Configuración	InterviewSetupForm, interviewSetupSchema (Zod)	Implementado
NER-11 / CU-05 Crear sesión	POST /api/interview/start, InterviewSession	Implementado
CU-06 Entrevista IA	useInterviewConversation, @elevenlabs/react	Implementado
CU-07 Finalizar/reporte	/end, elevenlabs-analysis, evaluations	Implementado
CU-09 Historial	/interview/reports, Prisma findMany por userId	Implementado
CU-10 Agendar	/api/interview/schedule, schedules, Resend	Implementado
Gestión de usuarios y roles	plugin admin de Better Auth, /admin, require-role	Implementado
Módulo de soporte	support_tickets, support_replies, /support, /admin/support	Implementado
Tiempo real del servidor	SSE en /api/realtime/*	Implementado
Personalización con CV	/api/cv, miniatura con pdfjs, contexto para el agente	Implementado
Límite de duración	20 minutos con aviso y cierre ordenado	Implementado

11. Conclusiones

La Fase 3 entrega un MVP funcional y ejecutable de Nervio alineado con la arquitectura y los prototipos de la Fase 2: acceso seguro con recuperación de contraseña, roles user/admin con gestión de usuarios, entrevistas por voz en tiempo real personalizadas con el CV y limitadas a 20 minutos, reportes con grabación y enlaces compartidos, historial, agendamiento y un módulo de soporte con actualizaciones en tiempo real por SSE. La implementación aplica arquitectura en capas, seguridad en la capa de aplicación (hashing, sesiones, roles, control de propiedad y validación con Zod), persistencia con migraciones e integridad referencial, una colección Postman que cubre toda la API y despliegue automatizado. Las pruebas automatizadas quedan para la Fase 4.

12. Bibliografía

Better Auth. (s. f.). Email & Password. https://better-auth.com/docs/authentication/email-password

Prisma Data, Inc. (s. f.). Prisma schema. https://www.prisma.io/docs/orm/prisma-schema

Vercel. (s. f.). Cron Jobs. https://vercel.com/docs/cron-jobs

OWASP. (s. f.). Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

ElevenLabs. (s. f.). Conversational AI. https://elevenlabs.io/docs/conversational-ai

13. Anexos

- **Repositorio:** colección de API en nervio-api.postman_collection.json (raíz del repo).
- **Seed y migraciones:** frontend/prisma/seed.ts y frontend/prisma/migrations/.
- **Documento maestro:** Fase 1 y Fase 2 en Actividad evaluada N.º 4 (este informe se anexa a continuación de dicho documento).